

Computer Vision

Neural Networks and CNNs: Architecting Deep Learning Systems for Visual Recognition

Section 1: Neural Network Foundations

1.1 Introduction to Neural Networks

- **Computational Model:** Inspired by the human brain, designed to approximate complex functions and separate non-linearly separable data.
- **Artificial Neurons:** The basic unit combining a linear function and an activation function (e.g., Sigmoid, ReLU).
- **Deep Neural Networks (DNN):** Architectures containing more than one hidden layer, allowing the model to learn high-level abstractions.
- **Learning Mechanism:** Parameters are optimized using Gradient Descent. However, as networks grow deeper, they face the Vanishing Gradient Problem.

1.2 Activation Functions and Optimization

- **Sigmoid Function:** Maps values to a 0-1 range; historically significant but prone to vanishing gradients in deep layers.
- **ReLU (Rectified Linear Unit):** Returns 0 for negative inputs and the identity for positive; the primary solution for the vanishing gradient problem.
- **Softmax Function:** Specifically used in the output layer for multi-class classification to convert raw scores into a probability distribution (summing to 1).
- **Overfitting Risks:** Adding too many layers or neurons can lead to models that memorize noise rather than learning general patterns.

1.3 Convolutional Neural Networks (CNN)

- **Spatial Awareness:** Unlike Fully Connected networks, CNNs preserve the spatial structure of images using kernels and weights.
- **Feature Hierarchy:** Automatically learns to detect low-level edges, mid-level textures, and high-level object concepts.
- **Parameter Efficiency:** Uses weight sharing and local connectivity to drastically reduce the number of learnable parameters compared to DNNs.

Section 2: Deep Learning Architectures

2.1 VGG Architecture (Oxford, 2014)

- **Core Philosophy:** Proved that depth with small filters (3x3) significantly improves classification accuracy.
- **Structure:** Consists of stacked 3x3 convolutions with a stride of 1 and Max-Pooling to reduce spatial dimensions.
- **Versions:** VGG16 (16 layers) and VGG19 (19 layers) are the most widely recognized implementations.
- **Architectural Trade-offs:** Extremely heavy (138M+ parameters), slow to train, and requires significant memory resources.

2.2 ResNet: Residual Networks (Microsoft, 2015)

- Residual Learning: Introduces 'Skip Connections' (Identity Mapping) that allow gradients to flow through the network without degradation.
- Solving Vanishing Gradients: By learning the residual function $F(x) = H(x) - x$, it enables the training of ultra-deep networks (e.g., ResNet-152).
- Accuracy and Optimization: Prevents the degradation problem where deeper networks actually perform worse than shallower ones.
- Industry Impact: ResNet is currently the backbone for most production-grade image recognition and GenAI vision tasks.

Section 3: Practical Implementation (Notebook Walkthrough)

Cell 1: Building a Fully Connected Layer with Activation

Initializing a linear layer and applying the ReLU activation function to solve non-linear problems.

```
import torch
import torch.nn as nn

# Defining a basic neuron logic
linear_layer = nn.Linear(in_features=784, out_features=128)
activation = nn.ReLU()

# Pass input through layer
x = torch.randn(1, 784)
output = activation(linear_layer(x))
```

Cell 2: Implementing Softmax for Multi-Class Output

Converting raw logits into standardized probabilities for architectural decision-making.

```
import torch.nn.functional as F

# Raw network output (logits)
logits = torch.tensor([2.0, 1.0, 0.1])
# Softmax calculation
probabilities = F.softmax(logits, dim=0)
print(f'Probabilities: {probabilities} | Sum: {torch.sum(probabilities)}')
```

Cell 3: Convolutional Layer and Max Pooling

Extracting spatial features while reducing dimensionality to prevent overfitting.

```
conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride=1, padding=1)
pool = nn.MaxPool2d(kernel_size=2, stride=2)

input_img = torch.randn(1, 3, 32, 32)
feature_map = pool(conv(input_img))
print(f'Feature Map Shape: {feature_map.shape}')
```

Cell 4: POC - Building a Residual Block

Architecting the skip connection logic to preserve gradient flow in deep networks.

```
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv = nn.Conv2d(channels, channels, 3, padding=1)

    def forward(self, x):
        # The Skip Connection:  $H(x) = F(x) + x$ 
        return F.relu(self.conv(x) + x)
```

Cell 5: Deep Learning Training Architecture

Implementing Gradient Descent and loss calculation for model parameter optimization.

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

# Standard iteration
optimizer.zero_grad()
output = model(X_train)
loss = criterion(output, y_train)
loss.backward() # Backpropagation
optimizer.step()
```